



اَوْنِوَرْسِيْتِي تِيْكْنُوْلُوْجِي مَآرَا
UNIVERSITI
TEKNOLOGI
MARA

ICT602

MOBILE TECHNOLOGY AND DEVELOPMENT

CHAPTER 3 - MOBILE DESIGN PRINCIPLES

Outcome



- ▶ Explain and discuss different screen sizes
- ▶ Explain and discuss challenges in supporting multiple screen sizes / resolution
- ▶ Calculate Screen Sizes/Resolution/DPI
- ▶ Describe Strategies to adapt Layout and Assets to Multiple Screen Sizes



Topic Outline

1. Different screen sizes
2. Challenges in supporting multiple screen sizes / resolution
3. Calculating Screen Sizes/Resolution/DPI
4. Designing for Multiple Screen Strategies



Different screen sizes



Why Different Screen sizes ?



- ▶ Not every mobile devices are equals.
 - ▶ Mobile devices come in all shapes and sizes. **Choice is great for consumers, but bad for designers.**
 - ▶ How to distinguish?
 - ▶ What is the type of your mobile device?



SMARTPHONE



PHABLET



TABLET

What are the Standards screen sizes for mobile devices?



- ▶ Standards screen sizes for mobile devices
 - ▶ 2.5", 2.75", 3.2", 4.2", 4.5", 4.75", 5", 5.25", 6", 6.5" 7", 9" and 10"
- Standard phone phablet tablet



Phablet



- ▶ Phablet is made up of two words **phone** + **tablet**.
- ▶ size of phablet is smaller than the tablets but bit larger than the smartphones
- ▶ contains the functions of tablets like as touchscreen, virtual keyboards, mobile operating system, camera, app storage and many more.
- ▶ Come with a stylus.

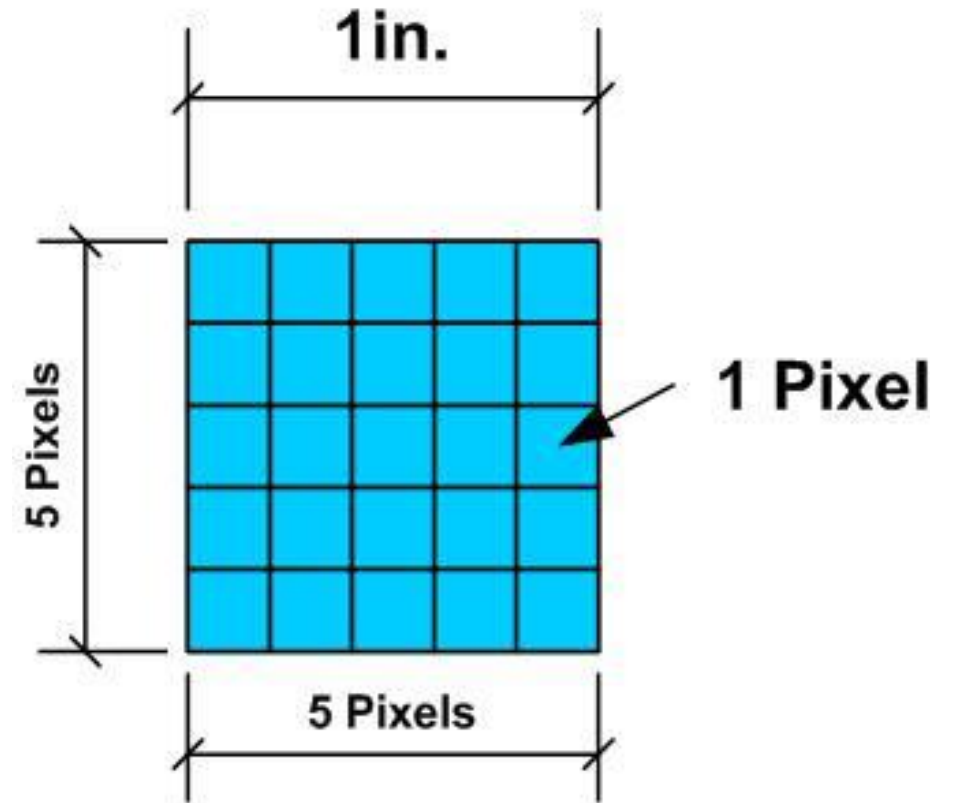


Why Different Screen sizes ?



- ▶ Even the sizes are same, these devices also would have **different screen densities**
- ▶ measured in “dots per-inch” (**dpi**)

Number of pixels
----- = ppi/dpi
screen length (inch)



**Resolution =
5 DPI (dots per inch)**

Challenges in supporting multiple screen sizes / resolution

Challenges in supporting multiple screen sizes



- ▶ Need to target the widest audience, while not alienating users who use larger/smaller devices
- ▶ Need to **identify the most common users target phone screen sizes**/density (different for each application)
- ▶ Etc.



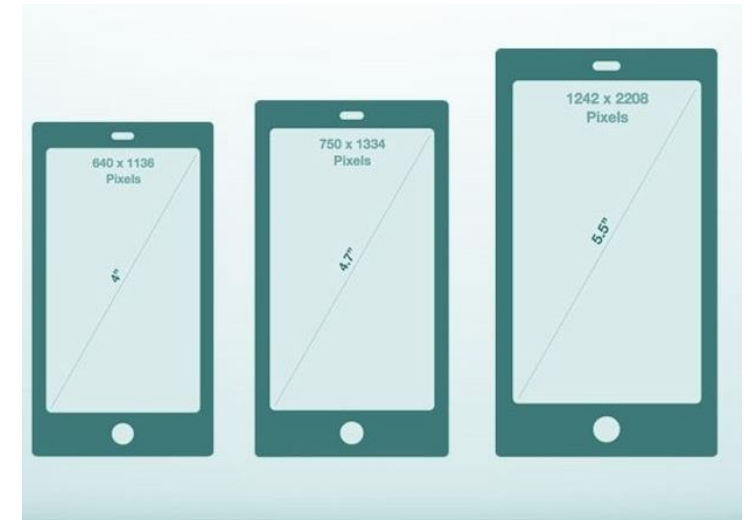
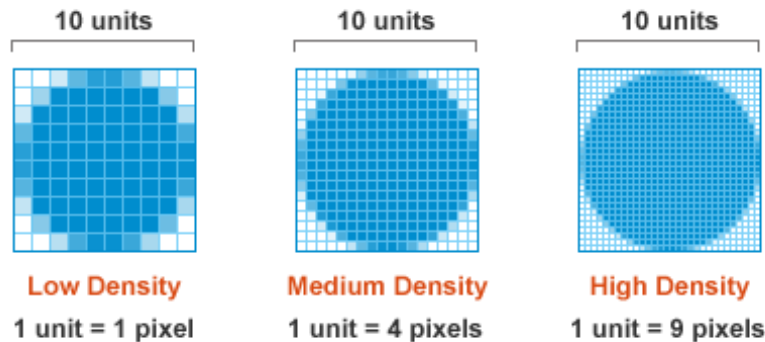
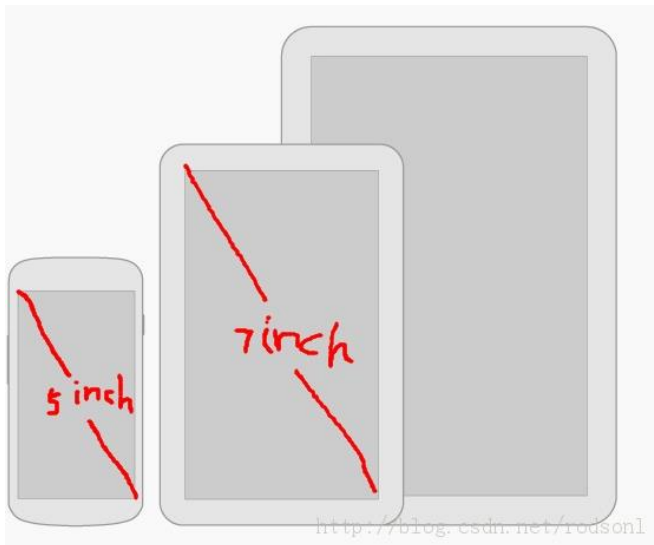
Size, Density, and Resolution



Screen size - amount of physical space available for displaying an interface, screen's diagonal, inch.

Screen density - how many pixels appear within a constant area of the display, pixel per inch = ppi

Screen resolution - number of pixels available in the display, scale-independent pixel = sp



Density Comparison



Standard Slate
10.6" 1366x768
148 PPI



HD Slate
10.6" 1920x1080
208 PPI

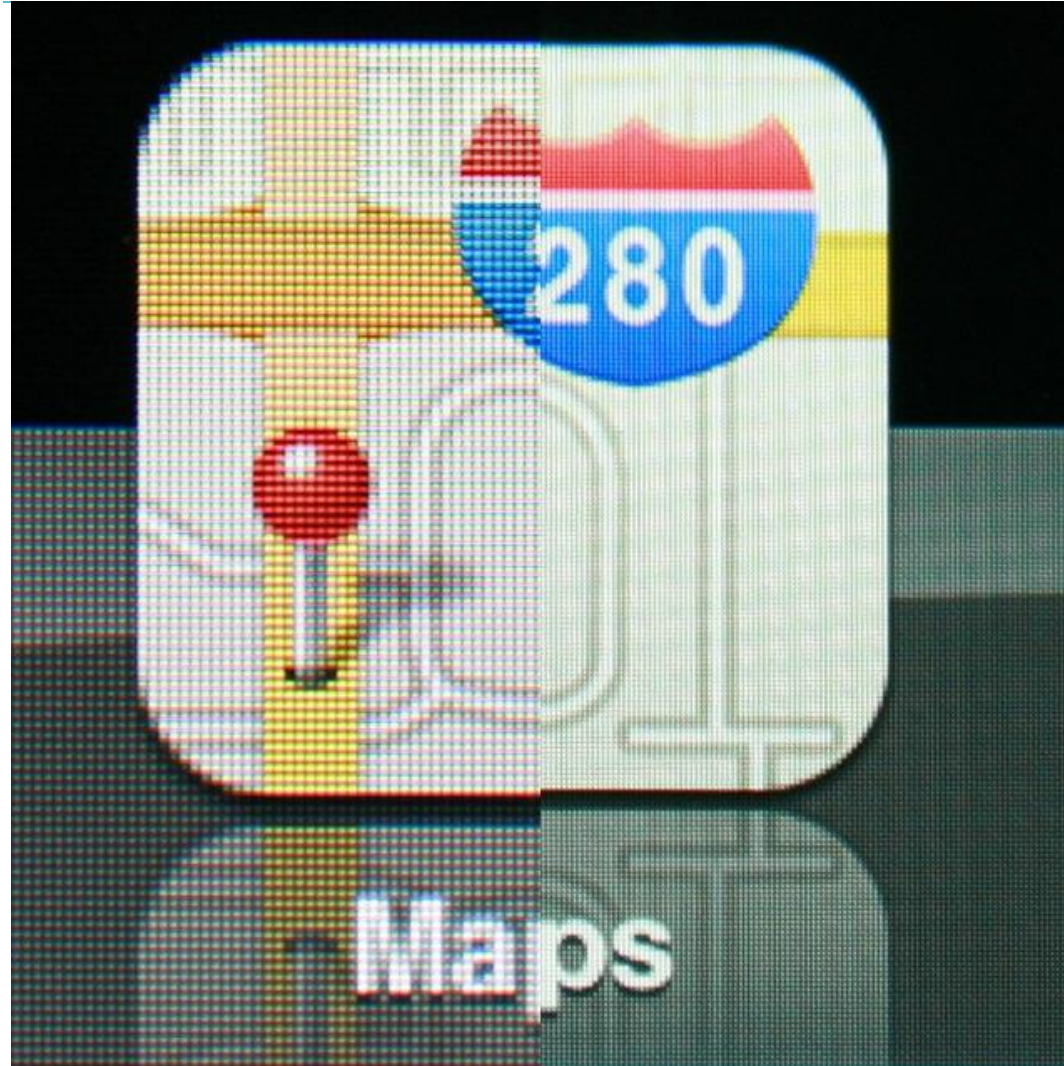
Density Comparison



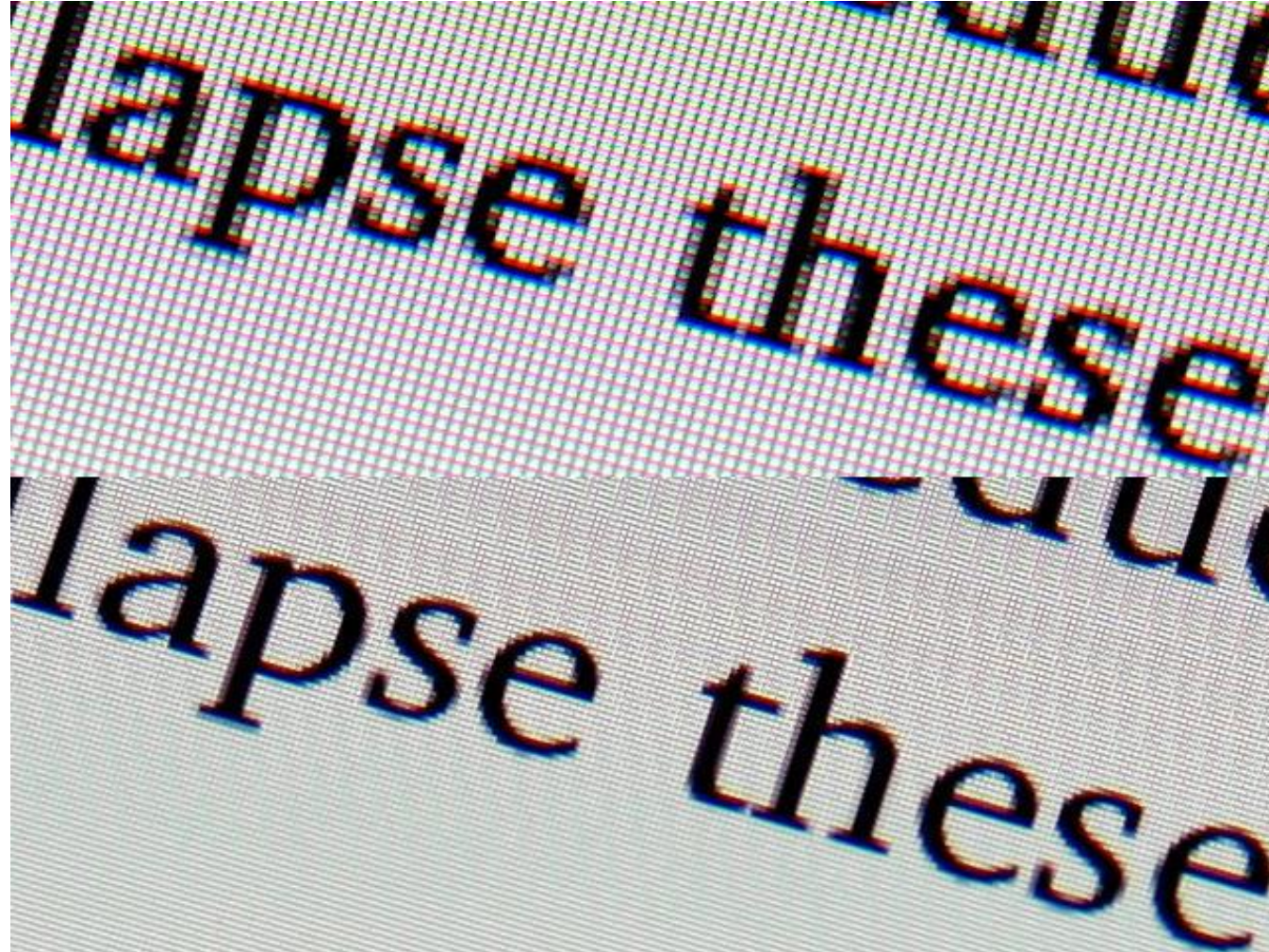
Apple iPhone 3GS: 165 ppi

Apple iPhone 4: 326 ppi

Density Comparison



Density Comparison



Screen sizes and Densities



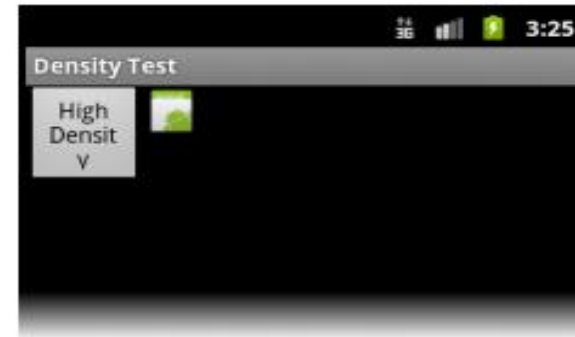
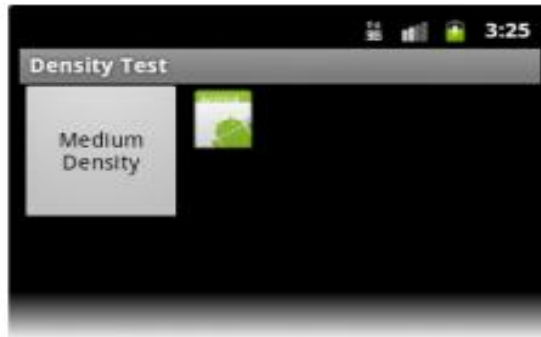
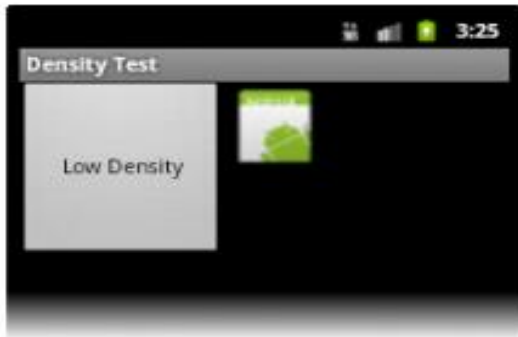
- ▶ Mobile devices usually have a **large variation of DPI** compared to their screen sizes.
- ▶ For example :
 - ▶ HTC One Max (5.9") has 373 dpi
 - ▶ Google Nexus 6 (5.9") has 493 dpi
 - ▶ HTC Pure (3.2") has 292 dpi
 - ▶ Samsung Galaxy Mini 2 (3.2") has 176 dpi
- ▶ This is unlike a computer monitor which almost always have a DPI between **72-140**



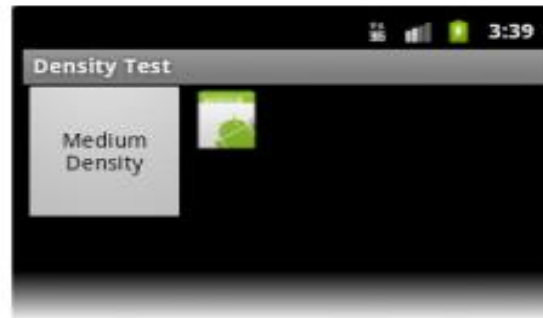
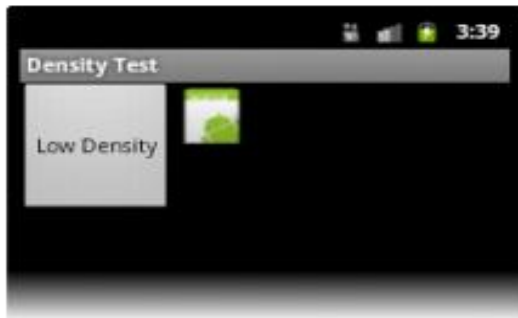
Designing for Screen Densities/Size Independence



- ▶ Differences of application without support for different densities and one with good support.



BAD!



GOOD!

Consistent Display on Multiple Devices



Good Example of Balancing Design on Different Devices

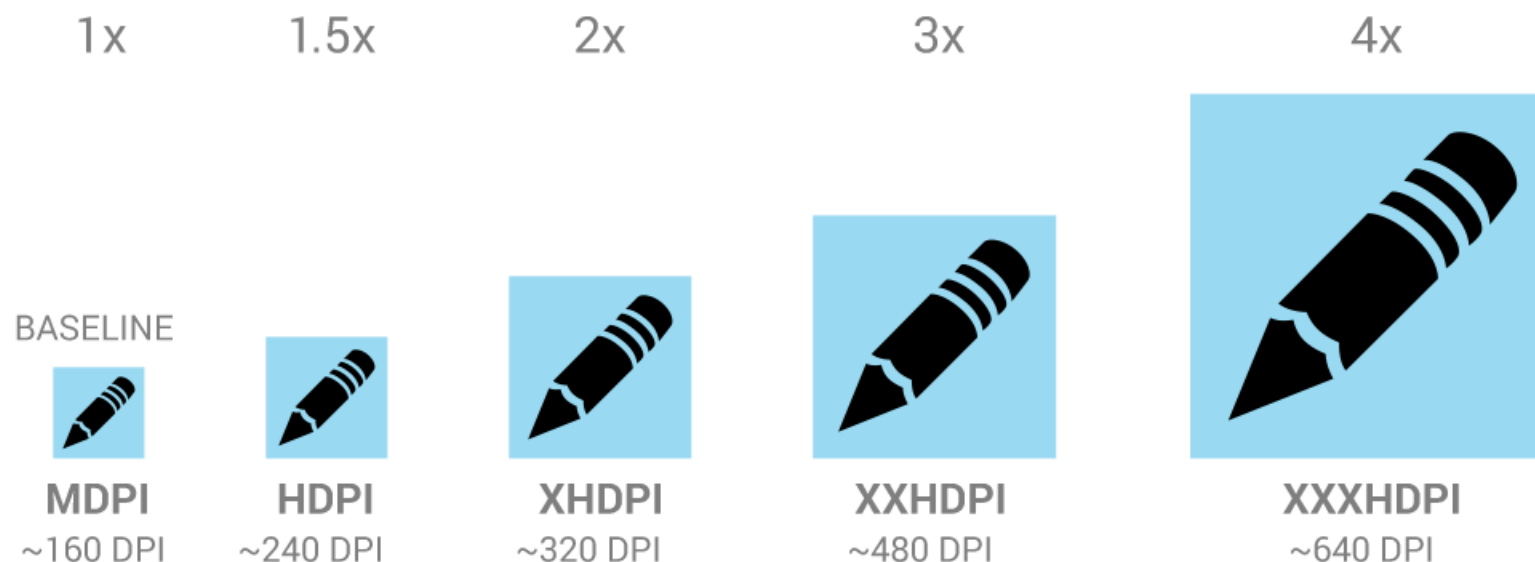






Screen sizes and Densities

- ▶ Therefore, an icon designed for Google Nexus 6 (445ppi) would be larger on Samsung Galaxy Mini 2! (176ppi)

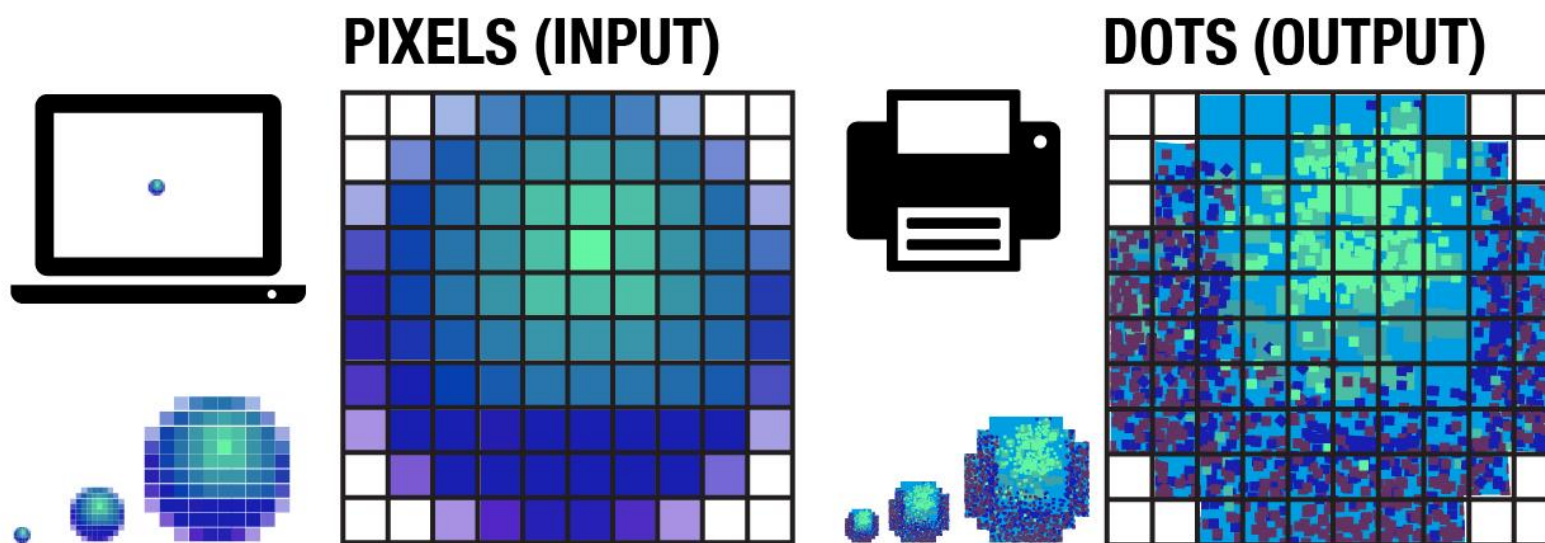


- ▶ A well design application would scale the UI design and graphic asset to match the screen size and densities.



DPI and PPI

- ▶ **PPI** refers to the number of pixels contained within one inch of an image displayed on a computer monitor.
- ▶ **DPI** refers to the number of printed dots contained within one inch of an image printed by a printer.



Calculating Screen Sizes / Resolution / DPI



How to calculate DPI

- ▶ Most Mobile devices manufacturers list their devices' "Screen sizes" and "Resolution" in their websites
- ▶ This is because of marketing decision.
 - ▶ Most consumers are more interested in the screen size and the screen resolution to view multimedia files and applications
- ▶ Dot per-inch is only used by developers to develop a portable and a sharper looking graphic asset for mobile application.

Model	Display Type & Size	Pixel Resolution	PPI (Pixel Per Inch)	Contrast Ratio	Max Brightness	Key Features
Samsung Galaxy S23 Ultra	6.8-inch Dynamic AMOLED 2X Display	3088 x 1440 pixels	500 ppi	Infinite (AMOLED)	1750 nits (peak)	120Hz, HDR10+, Always-On Display, Vision Booster
Samsung Galaxy S23+	6.6-inch Dynamic AMOLED 2X Display	2340 x 1080 pixels	393 ppi	Infinite (AMOLED)	1750 nits (peak)	120Hz, HDR10+, Always-On Display, Vision Booster
Samsung Galaxy S23	6.1-inch Dynamic AMOLED 2X Display	2340 x 1080 pixels	422 ppi	Infinite (AMOLED)	1750 nits (peak)	120Hz, HDR10+, Always-On Display, Vision Booster
Samsung Galaxy Z Fold 5	7.6-inch Foldable Dynamic AMOLED 2X (Main Display)	2176 x 1812 pixels	374 ppi	Infinite (AMOLED)	1750 nits (peak)	120Hz, HDR10+, Always-On Display
Samsung Galaxy Z Flip 5	6.7-inch Foldable Dynamic AMOLED 2X Display	2640 x 1080 pixels	426 ppi	Infinite (AMOLED)	1600 nits (peak)	120Hz, HDR10+, Always-On Display

Model	Display Type & Size	Pixel Resolution	PPI (Pixel Per Inch)	Contrast Ratio	Max Brightness	Key Features
iPhone 15 Pro Max	6.7-inch Super Retina XDR Display with ProMotion	2796 x 1290 pixels	460 ppi	2,000,000:1	2000 nits (outdoor) / 1600 nits (HDR)	ProMotion (120Hz), Always-On Display, Dynamic Island
iPhone 15 Pro	6.1-inch Super Retina XDR Display with ProMotion	2556 x 1179 pixels	460 ppi	2,000,000:1	2000 nits (outdoor) / 1600 nits (HDR)	ProMotion (120Hz), Always-On Display, Dynamic Island
iPhone 15 Plus	6.7-inch Super Retina XDR Display	2796 x 1290 pixels	460 ppi	2,000,000:1	2000 nits (outdoor) / 1600 nits (HDR)	HDR10, Dolby Vision
iPhone 15	6.1-inch Super Retina XDR Display	2556 x 1179 pixels	460 ppi	2,000,000:1	2000 nits (outdoor) / 1600 nits (HDR)	HDR10, Dolby Vision
iPhone 14 Pro Max	6.7-inch Super Retina XDR Display with ProMotion	2796 x 1290 pixels	460 ppi	2,000,000:1	2000 nits (outdoor) / 1600 nits (HDR)	ProMotion (120Hz), Always-On Display, Dynamic Island
iPhone 14 Pro	6.1-inch Super Retina XDR Display with ProMotion	2556 x 1179 pixels	460 ppi	2,000,000:1	2000 nits (outdoor) / 1600 nits (HDR)	ProMotion (120Hz), Always-On Display, Dynamic Island





How to calculate Dot per-inch (DPI)

- ▶ Given Device X has,

Diagonal Screen Size, **D**: 6"

Resolution, **r**: 2560x1440

- ▶ Then you need to **①calculate screen aspect ratio**, from Resolution, using greatest common denominator (**GCD**):

2560x1440, GCD of 2560 and 1440 is **160**

So, $2560/160 = 16$, $1440/160 = 9$. Thus the screen ratio of 2560x1440 is **16:9** (we label it as '**R**' = L:W)

How to calculate Dot per-inch (DPI)



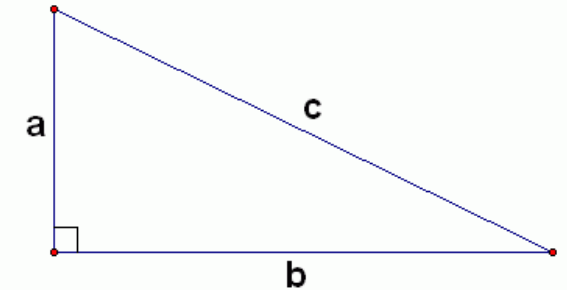
- Afterwards you need to use Pythagoras Theorem to
② **calculate screen height and length** from diagonal size. $D = 6''$

$$\begin{aligned} D' &= d / D \\ &= 18.35 / 6'' \\ &= \mathbf{3.06} \end{aligned}$$

$$\begin{aligned} L' &= L / 3.06 \\ &= 16 / 3.06 \\ &= \mathbf{5.23''} \end{aligned}$$

$$\begin{aligned} d^2 &= L^2 + W^2, \text{ screen ratio } 16:9 \\ d^2 &= 16^2 + 9^2 \\ d^2 &= 256 + 81 \\ d^2 &= 337, d = \mathbf{18.35} \end{aligned}$$

$$\begin{aligned} W' &= W / 3.06 \\ W' &= 9 / 3.06 \\ W' &= \mathbf{2.94''} \end{aligned}$$



$$a^2 + b^2 = c^2$$

Screen length = 5.23''
Screen Width = 2.94''



How to calculate Dot per-inch (DPI)

▶ Get Screen Resolution, $r = 2560 \times 1440$

▶ $rl = 2560$, $rw = 1440$

$DPI = rl / L' = 2560 / 5.23 = \mathbf{489}$ (round to nearest int)

$DPI = rw / W' = 1440 / 2.94 = \mathbf{489}$ (round to nearest int)

Thus the DPI for Device X is **489 ± 1**

Exercise



- ▶ Calculate DPI for
 - ▶ Device A – Screen size: 4.7", Res: 1080x1920
 - ▶ Device B – Screen Size: 5", Res: 1920x1080
 - ▶ Device C – Screen size: 3.2", Res: 480x800
- ▶ Determine whether Device A, B, and C is MDPI, HDPI, XHDPI, XXHDPI, or XXXHDPI.



Answer

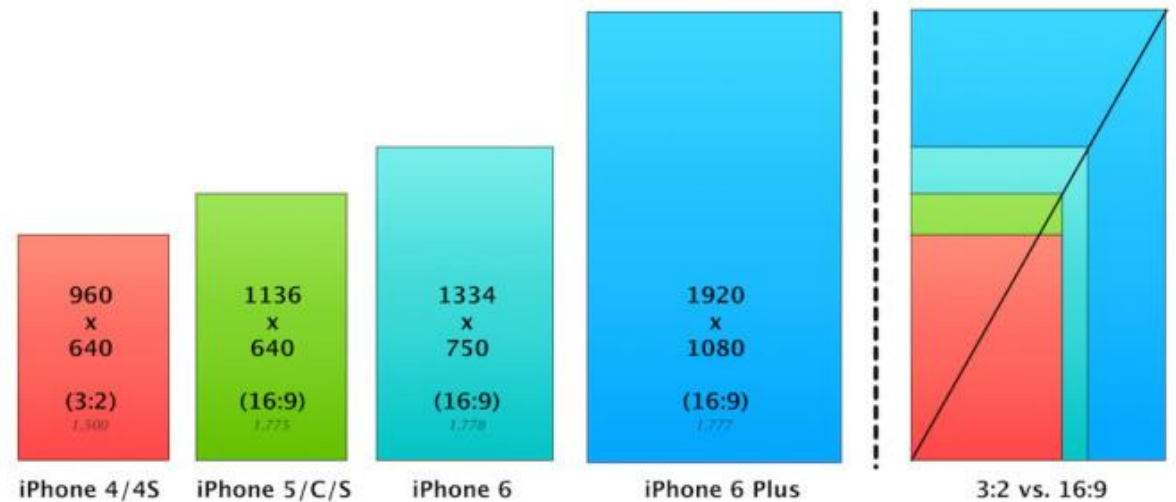
- ▶ Device A = 468 ± 1 (XXHDPI)
- ▶ Device B = 440 ± 1 (XXHDPI)
- ▶ Device C = 291 ± 1 (XHDPI)
- ▶ You can re-check your answer in <http://dpi.lv> website, answer is always ± 1 due to rounding error

Hint



- ▶ Most mobile device has either 16:9 or 3:2 screen aspect ratio, however some mobile devices also adopt 16:10 (8:5) and 13.5:9 aspect ratio.

5:4 Computer Displays	4:3 SDTV / Video Computer Displays	3:2 35mm Film DSLR Cameras Smartphones	16:10 Widescreen Computer Displays Smartphones
16:9 HDTV Widescreen SDTV Smartphones	1.85:1 Cinema Film (US)	2.35:1 Cinemascope	



- ▶ Calculation for **DPI must be equal for r_l/L' and r_w/W'** . If it is not equal, then there's an error in your calculation. You can use this to ensure if you got the right answer.



Strategies for Multiple Screen Design

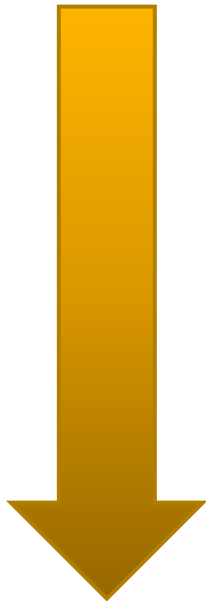




Designing for Multiple Screen Strategies

- ▶ Mobile content developers has adapted several **strategies** for supported multiple screens sizes/densities

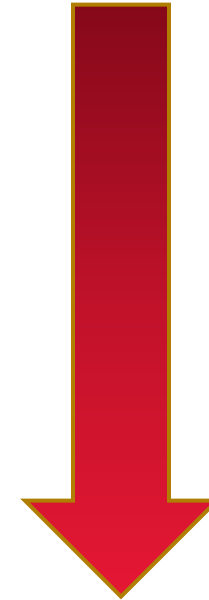
Easier



Harder

1. Designing app for the most common DPI / Screen Sizes
2. Restricting app only for certain DPI / Screen Sizes
3. Including multiple graphic assets for different screen sizes/DPI
4. Accommodating different screen sizes programmatically

Limited Reach



Wider Reach

1st Strategy: Designing app for the most common DPI / Screen Sizes



- ▶ This is the **easiest** strategy for designing mobile application UI
- ▶ In this strategy, the developers will do a **survey** to determine which are the **most common DPI and screen sizes** most commonly use by their users
- ▶ Then, the developers will proceed optimizing the application for the most common screen size/DPI.
- ▶ This approach will ignore other screen sizes/DPI as 'minorities'

1st Strategy: Designing app for the most common DPI / Screen Sizes



Advantages:

- ▶ Development and prototyping will be **faster**, because development will **ignore** other screen sizes/resolution
- ▶ **Easiest** to follow, does not waste any time optimizing for different screen sizes.
- ▶ Only targets the majority users devices

1st Strategy: Designing app for the most common DPI / Screen Sizes



Weaknesses:

- ▶ There are varieties of mobile devices that has vastly different screen size and ratios
- ▶ Some users not in the target devices group may find the application **poorly displayed** on their mobile phone screen. Certain graphics would be too large or too small. Thus may result in **poor user perception and experience**
- ▶ Application may look '**amateurish**' and poorly designed
- ▶ Standards of mobile device screen size and resolution will change over time. Application that is 'locked' in certain screen size/resolution may **get outdated quickly**

2nd Strategy: Restricting app only for certain DPI / Screen Sizes



- ▶ This approach is similar to “Designing app for the most common DPI”
- ▶ However this approach adds another layer of **enforcement** in the application programming which **prevents it to be executed in devices which has (commonly) lower DPI.**
- ▶ Users of devices with lower DPI will not be able to run the application.

2nd Strategy: Restricting app only for certain DPI / Screen Sizes



Advantages

- ▶ Development and prototyping will be **faster**, because development will focus on devices with higher DPI
- ▶ Only targets higher-end devices while ignoring devices with lower DPI

Weaknesses:

- ▶ **Limited audience**. Some users with lower DPI or less popular device will not be able to execute application

3rd Strategy: Including multiple graphic assets for different screen sizes/DPI



- ▶ Design only a **single application**, but includes multiple graphic assets and layout to **support different screen size & DPI**
- ▶ This approach is officially supported by Android official documentation. iOS on the other hand offers devices with standard screen size and DPI
- ▶ This approach requires developers to create different graphic assets for different screen densities (DPI) while maintaining a single application
- ▶ **Single application -> multiple graphics**

3rd Strategy: Including multiple graphic assets for different screen sizes/DPI



- ▶ Using this approach developer will create several graphics with different size to support different screen densities.

(144x144)

(96x96)

(72x72)

(48x48)



xxhdpi



xhdpi



hdpi



mdpi

- ▶ This will display approximately the same size (but with extra detail) on mobile devices with different resolution/DPI



Name

Date modified

drawable	7/5/2015 4:48 AM
drawable-hdpi	7/5/2015 4:48 AM
drawable-mdpi	7/5/2015 4:48 AM
drawable-xhdpi	7/5/2015 4:48 AM
drawable-xxhdpi	7/5/2015 4:48 AM
layout	7/5/2015 5:59 AM
menu	7/5/2015 4:48 AM
values	7/5/2015 4:48 AM
values-w820dp	7/5/2015 4:48 AM

- main
 - assets
 - java
 - res
 - drawable-hdpi
 - drawable-mdpi
 - drawable-xhdpi
 - layout
 - menu
 - mipmap-hdpi
 - mipmap-mdpi
 - mipmap-xhdpi
 - mipmap-xxhdpi
 - mipmap-xxxhdpi
 - values
 - AndroidManifest.xml

3rd Strategy: Including multiple graphic assets for different screen sizes/DPI



- ▶ Android categorizes Screen Densities in several overlapping group
 - i. ldpi = 120 dpi or less
 - ii. mdpi = between 120-200 dpi
 - iii. hdpi = between 200 – 300 dpi
 - iv. xhdpi = between 300-400 dpi
 - v. xxhdpi = more than 400 dpi
- ▶ Developers need to create graphic asset for each of the densities category

3rd Strategy: Including multiple graphic assets for different screen sizes/DPI



Advantages

- ▶ Allows developers to develop a single application for multiple device
- ▶ Application can **support widest users** devices as possible

Disadvantages

- ▶ Creating multiple graphic assets can **consume time**
- ▶ May have to create graphic assets to cater for uncommon device

4th Strategy: Accommodating screen sizes programmatically different



- ▶ This approach is usually used by Mobile Web Application
- ▶ Instead of relying with hardcoded width and length, developers instead **uses percentages** to describe the device layout
- ▶ This strategy also commonly combined with Android approach of “Including multiple graphic assets for different screen sizes/DPI”
- ▶ Sometimes this approach also uses Scalable Vector Graphic (**SVG**) to **scale graphics asset programmatically** without sacrificing quality.

Accommodating screen sizes programmatically different



Advantages

- ▶ Allows developers to develop a **single application for multiple device**
- ▶ Application can **support widest users** devices as possible

Disadvantages

- ▶ Usually only supported in Mobile Web Application
- ▶ May have to create graphic assets to cater for uncommon device



End of Lecture ...

